PCEP-30-03 · AMALIY TEST TAHLILI

Python asoslari: savollar va mavzular

Test savollari va ularga oid mavzular — oddiy tilda, misollar bilan

9 ta savol | 8 ta mavzu | kod misollari

REJA

Bugun 8 ta mavzuni ko‘rib chiqamiz

1
Dasturlash tili tuzilishi
1-savol

2
Operatorlar
5-savol

3
Tuple va list
18-savol

4
Set (to‘plam)
19-savol

5
Satrlarni formatlash
23-savol

6
Istisnolar (xatolar)
29-savol

7
Fayllar: with va csv
31 va 32-savol

8
random moduli
35-savol

Dasturlash tilining to‘rt tushunchasi

Lexis

so‘zlar lug‘ati

Tilda qanday so‘z va belgilar bor: kalit so‘zlar, nomlar, sonlar. Masalan: **print**, **if**, **+**.

Syntax

yozish qoidalari

So‘zlarni qanday tartibda yozish kerak. **if x > 5:** — to‘g‘ri, **if > x 5:** — xato.

Semantics

ma’no

Yozilgan gap nimani anglatadi. **5 + "a"** — yozilishi to‘g‘ri, lekin ma’nosi yo‘q.

Interpretation

bajarish usuli

Kod qanday ishga tushadi. Python kodni qatorma-qator o‘qib bajaradi.

Inson tiliga o‘xshatamiz: lug‘at (lexis) → grammatika (**syntax**) → ma’no (semantics).

SAVOL 1 · TIL TUZILISHI · 1 TA JAVOB

Which programming language concept defines the rules for arranging valid language elements into correctly structured statements?

Qaysi tushuncha to‘g‘ri til elementlarini to‘g‘ri tuzilgan gaplarga joylash qoidalarini belgilaydi?

Eslab qoling

Syntax — elementlarni qanday tartibda yozish qoidalari. Lexis — so‘zlar, semantics — ma’no, interpretation — bajarish.

A **Interpretation**
Noto‘g‘ri: bu kodni qanday bajarishga tegishli

B **Syntax**
To‘g‘ri: aynan gap tuzish qoidalari

C **Semantics**
Noto‘g‘ri: bu gapning ma’nosi haqida

D **Lexis**
Noto‘g‘ri: bu faqat so‘zlar lug‘ati

Arifmetik operatorlar

Operator	Nomi	Misol	Natija
+	Qo‘shish	7 + 2	9
-	Ayirish	7 - 2	5
*	Ko‘paytirish	7 * 2	14
/	Bo‘lish (natija float)	7 / 2	3.5
//	Butun qismigacha bo‘lish	7 // 2	3
%	Bo‘lishdan qoldiq	7 % 2	1
**	Darajaga ko‘tarish	7 ** 2	49

Unar operator, / va // farqi

Unar va binar

Unar operator bitta operand oladi: **-5**. Binar operator ikkita operand oladi: **5 - 3**.

/ va // bir xil emas

/ har doim float beradi: **6 / 3** → **2.0**. **//** butun qismini beradi: **7 // 2** → **3**.

```
print(-5)          # unar minus: 1 operand
print(5 - 3)       # binar minus: 2 operand
print(6 / 3)       # 2.0 (float)
print(7 // 2)      # 3
print(-7 // 2)     # -4 (pastga yaxlitlanadi)
print(7 % 2)       # 1
print(2 ** 3)      # 8
```

SAVOL 5 · OPERATORLAR · 2 TA JAVOB TANLANG

Which two statements about Python operators are correct?

Python operatorlari haqidagi qaysi ikkita gap to‘g‘ri?

Eslab qoling

% — bo‘lishdan qoldiq (7 % 2 = 1). **** — daraja (2 ** 3 = 8). **/** har doim float, **//** esa butun qismini beradi.

A **A unary – operator requires two operands.**
Noto‘g‘ri: unar operator bitta operand oladi

B **The % operator can be used to obtain the remainder from a division.**
To‘g‘ri: qoldiqni beradi: 7 % 2 = 1

C **The ** operator performs exponentiation.**
To‘g‘ri: darajaga ko‘tarish: 2 ** 3 = 8

D **The // operator always produces the same result as /.**
Noto‘g‘ri: 7 / 2 = 3.5, 7 // 2 = 3

E **The / operator always produces an integer result.**
Noto‘g‘ri: / float beradi: 6 / 3 = 2.0

List va tuple: asosiy farq

Xususiyat	list	tuple
Qavs turi	kvadrat qavs **[ ]**	oddiy qavs **( )**
Elementni o‘zgartirish	**mumkin (mutable)**	**mumkin emas (immutable)**
Indeks bilan olish	mumkin: **a[0]**	mumkin: **t[0]**
Turli tipdagi qiymatlar	mumkin	mumkin
Ichida boshqa to‘plam	mumkin	mumkin

Asosiy farq — o‘zgaruvchanlik: tuple yaratilgandan keyin uni o‘zgartirib bo‘lmaydi.

Tuple yaratish va ishlatish

```
t = (10, "olma", 3.5)      # turli tipdagi qiymatlar
print(t[0])                # 10: indeks ishlaydi
t[1] = "nok"               # TypeError (xato)

one = (5,)                 # 1 elementli tuple
nested = ([1, 2], (3, 4)) # ichida list va tuple
```

Tuple o‘zgarmas

Yaratilgach elementni almashtirib yoki o‘chirib bo‘lmaydi.

Indeks ishlaydi

t[0] kabi indeks bilan olish mumkin — list’dagidek.

Bitta element

(5,) — vergul bo‘lmasa bu oddiy son bo‘lib qoladi.

SAVOL 18 · TUPLE VA LIST · 1 TA JAVOB

Which statement correctly compares tuples and lists?

Qaysi gap tuple va list'ni to'g'ri solishtiradi?

Eslab qoling

Tuple — o'zgarmas (immutable): elementini almashtirib bo'lmaydi. List esa o'zgaruvchan (mutable).

A **Lists can contain only one data type, while tuples can contain several.**
Noto'g'ri: ikkalasida ham turli tip bo'ladi

B **Tuples support indexing, while lists do not.**
Noto'g'ri: indeks ikkalasida ham: a[0], t[0]

C **Tuples are immutable, while lists allow their elements to be modified.**
To'g'ri: tuple o'zgarmas, list o'zgaradi

D **Tuples cannot contain other collections.**
Noto'g'ri: ichida list, tuple, set bo'lishi mumkin

Set — noyob qiymatlar to‘plami

```
s = {1, 2, 2, 3}      # takror bor, xato yo‘q
print(s)              # {1, 2, 3}
print(2 in s)         # True
print(type({}))       # <class 'dict'>
print(type(set()))    # <class 'set'>
# s[0] → TypeError: indeks yo‘q
```

Noyob elementlar

Takrorlar bitta bo‘lib qoladi, xato yo‘q.

Indeks yo‘q

Tartib yo‘q: **s[0]** ishlamaydi.

in bilan tekshirish

3 in s → **True** yoki **False**.

Bo‘sh set

Bo‘sh set — **set()**. **{}** esa dict.

SAVOL 19 · SET · 2 TA JAVOB TANLANG

Which two statements about sets are correct?

Set'lar haqidagi qaysi ikkita gap to'g'ri?

Eslab qoling

Set — noyob (takrorlanmas) elementlar to'plami. Indeks yo'q, lekin **in** bilan tekshirish mumkin. Bo'sh set: set().

A **Sets support positional indexing with data[0].**
Noto'g'ri: set'da indeks yo'q — TypeError

B **The in operator can test whether a value belongs to a set.**
To'g'ri: 3 in s → True yoki False

C **A set stores unique elements.**
To'g'ri: takroriy qiymat bitta bo'lib qoladi

D **{} creates an empty set.**
Noto'g'ri: {} — bo'sh dict. Bo'sh set: set()

E **A duplicate value cannot appear in a set literal without causing an error.**
Noto'g'ri: xato bermaydi, takror olib tashlanadi

Formatlashning uchta usuli

Berilgan: **name = "Mia"** va **age = 20**

Usul	Kod	Natija
f-string (eng qulayi)	`f"{name} is {age}"`	`Mia is 20`
format() usuli	`"{} is {}".format(name, age)`	`Mia is 20`
% usuli (eski)	`"%s is %d" % (name, age)`	`Mia is 20`

Uchalasida natija bir xil. f-string'da qator oldidan **f** harfi, **{ }** ichida esa o'zgaruvchi nomi yoziladi.

Xatoga olib boradigan variantlar

`"{name} is {age}"`

Natija: {name} is {age}

f harfi yo‘q, shuning uchun matn o‘zicha chiqadi.

`f"name is age"`

Natija: name is age

{ } yo‘q, o‘zgaruvchi qiymati qo‘yilmaydi.

`"%d is %s" % (name, age)`

Natija: TypeError

%d son kutadi, **name** esa satr ("Mia").

To‘g‘ri variantlar: f"{name} is {age}" va "{} is {}".format(name, age).

SAVOL 23 · SATRLARNI FORMATLASH · 2 TA JAVOB TANLANG

Given name = "Mia" and age = 20, which two expressions produce Mia is 20?

name = "Mia" va age = 20 bo'lsa, qaysi ikkita ifoda «Mia is 20» chiqaradi?

Eslab qoling

f-string: f"{name} is {age}". format(): "{} is {}".format(name, age). **%d** son uchun, **%s** satr uchun.

A `"{name} is {age}"`
Noto'g'ri: f harfi yo'q — matn o'zicha chiqadi

B `f"name is age"`
Noto'g'ri: { } yo'q — qiymat qo'yilmaydi

C `"%d is %s" % (name, age)`
Noto'g'ri: %d son kutadi, name esa satr: TypeError

D `"{} is {}".format(name, age)`
To'g'ri: Mia is 20 chiqadi

E `f"{name} is {age}"`
To'g'ri: Mia is 20 chiqadi

Istisno nima va qanday turlari bor

Sintaksis xato

Kod yozilishidagi xato. Dastur umuman ishga tushmaydi. Masalan: yopilmagan qavs.

Istisno (exception)

Dastur ishlayotgan paytda chiqadigan xato. Masalan: **5 / 0**.

Istisno turi	Qachon chiqadi
`ZeroDivisionError`	nolga bo‘lish: **5 / 0**
`ValueError`	noto‘g‘ri qiymat: **int("abc")**
`TypeError`	tip mos emas: **"a" + 1**
`IndexError`	list indeksi yo‘q
`KeyError`	dict kaliti yo‘q
`NameError`	nom e’lon qilinmagan

try / except va ushlanmagan istisno

```
try:
    n = int("abc")
except ValueError:
    print("Bu son emas!")
print("Dastur davom etadi")

n = int("abc")        # try yo‘q: ValueError
print("Bu qatorga yetmaydi")
```

Ushlangan istisno

except bloki xatoni ushlaydi, dastur davom etadi.

Ushlanmagan istisno

Dastur oqimi to‘xtaydi, keyingi qatorlar bajarilmaydi.

Xato turi muhim

Har bir muammo o‘z istisno turini beradi.

SAVOL 29 · ISTISNOLAR · 2 TA JAVOB TANLANG

Which two statements about exceptions are correct?

Istisnolar haqidagi qaysi ikkita gap to‘g‘ri?

Eslab qoling

Istisno — dastur ishlayotganda chiqadigan xato. Turi muammoga bog‘liq. Ushlanmasa, dastur oqimi to‘xtaydi.

A **Python automatically ignores all exceptions and continues with the next statement.**
Noto‘g‘ri: e’tiborsiz qoldirmaydi — dastur to‘xtaydi

B **Exceptions can occur only when Python source code contains a syntax error.**
Noto‘g‘ri: kod ishlayotganda ham chiqadi: 5 / 0

C **Different runtime problems can produce different built-in exception types.**
To‘g‘ri: ZeroDivisionError, ValueError...

D **IndexError and KeyError always describe the same problem.**
Noto‘g‘ri: Index — list xatosi, Key — dict xatosi

E **An unhandled exception interrupts the normal flow of program execution.**
To‘g‘ri: dastur to‘xtaydi, traceback chiqadi

with bilan fayl o‘zi yopiladi

with’siz — eski usul

```
f = open("a.txt", "w")
f.write("Salom")
f.close()   # unutilsa, fayl ochiq qoladi
```

with bilan — tavsiya etilgan usul

```
with open("a.txt", "w") as f:
    f.write("Salom")
# blok tugadi: fayl o‘zi yopildi
```

Xavfsiz

Blok ichida xato chiqsa ham fayl yopiladi.

Qulay

close() ni qo‘lda yozish shart emas.

SAVOL 31 · WITH OPERATORI · 1 TA JAVOB

Why is the with statement commonly used when working with files?

*Fayllar bilan ishlaganda nega **with** operatori ko‘p ishlatiladi?*

Eslab qoling

with bloki tugashi bilan fayl avtomatik yopiladi, xato chiqsa ham. close() ni qo‘lda yozish shart emas.

A **It guarantees that every file operation executes faster.**
Noto‘g‘ri: tezlikka ta’siri yo‘q

B **It prevents any exception from occurring during file access.**
Noto‘g‘ri: xatolarning oldini olmaydi

C **It manages the file context and closes the file automatically when the block ends.**
To‘g‘ri: blok tugashi bilan fayl yopiladi

D **It allows every file to be read and written without specifying a mode.**
Noto‘g‘ri: rejim kerak; odatiy rejim — faqat o‘qish

csv moduli: qator yozish va o‘qish

Yozish

```
import csv

with open("data.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Ana", 90])    # 1-qator
    writer.writerow(["Bo", 80])     # 2-qator
```

O‘qish

```
with open("data.csv", newline="") as f:
    for row in csv.reader(f):
        print(row)    # ['Ana', '90']
```

writerow()

Ro‘yxatni bitta qator qilib yozadi: **Ana,90**.

Maydonlar

Ro‘yxat elementlari — maydonlar, vergul bilan ajratiladi.

newline=""

Qatorlar orasida ortiqcha bo‘sh qator chiqmaydi.

SAVOL 32 · CSV FAYLLAR · 1 TA JAVOB

What does the following code write as one CSV row?

Quyidagi kod CSV faylga qanday qator yozadi?

```
import csv

with open("data.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Ana", 90])
```

Eslab qoling

writerow() ro‘yxatni bitta qator qiladi, elementlar esa maydon bo‘ladi: Ana,90.

A One row containing the fields Ana and 90
To‘g‘ri: bitta qator, ikkita maydon

B The code reads one row instead of writing it.
Noto‘g‘ri: bu yerda yozish (writerow)

C Two separate rows containing Ana and 90
Noto‘g‘ri: ikki qator emas, bitta

D One row containing only the field Ana
Noto‘g‘ri: 90 ham yoziladi

random: pseudo-tasodifiy qiymatlar

```
import random

print(random.random())           # 0.0 ⩽ x < 1.0
print(random.randint(1, 6))      # butun son 1..6
print(random.choice([1, 2, 3]))  # element tanlash

random.seed(42)                  # boshlang‘ich holat
print(random.random())           # har safar bir xil
```

Pseudo-tasodifiy

Qiymatlar formula bilan hisoblanadi — haqiqiy tasodif emas.

random()

Float qaytaradi: 0.0 dan 1.0 gacha, 1.0 kirmaydi.

seed()

Boshlang‘ich holat. Bir xil seed — bir xil ketma-ketlik.

SAVOL 35 · RANDOM MODULI · 2 TA JAVOB TANLANG

Which two statements about pseudorandom generation are correct?

Pseudo-tasodifiy generatsiya haqidagi qaysi ikkita gap to‘g‘ri?

Eslab qoling

random() — 0.0 dan 1.0 gacha float (1.0 kirmaydi). **seed()** boshlang‘ich holatni beradi: bir xil seed — bir xil ketma-ketlik.

A **Using seed() converts pseudorandom generation into true physical randomness.**
Noto‘g‘ri: seed haqiqiy tasodif bermaydi

B **The argument passed to seed() becomes the next value returned by random().**
Noto‘g‘ri: seed — holat, qaytadigan qiymat emas

C **Calling random() always returns an integer.**
Noto‘g‘ri: random() float qaytaradi

D **random() produces a floating-point value from 0.0 up to, but not including, 1.0.**
To‘g‘ri: oraliq: 0.0 <= x < 1.0

E **Setting the same seed can be used to reproduce the same pseudorandom sequence.**
To‘g‘ri: bir xil seed — bir xil ketma-ketlik

XULOSA

To‘g‘ri javoblar bir qarashda

Savol	Mavzu	To‘g‘ri javob
1	Til tuzilishi	Syntax
5	Operatorlar	**%** (qoldiq) va **** (daraja)
18	Tuple va list	Tuple o‘zgarmas, list o‘zgaradi
19	Set	**in** operatori; set noyob elementlarni saqlaydi
23	Formatlash	**"{} is {}".format(name, age)** va **f"{name} is {age}"**
29	Istisnolar	Xato turlari har xil; ushlanmasa — dastur to‘xtaydi
31	with	Kontekstni boshqaradi, blok tugagach faylni yopadi
32	csv	Bitta qator: **Ana** va **90** maydonlari
35	random	**random()** — [0.0, 1.0); bir xil seed — bir xil natija

IMTIHON UCHUN MASLAHAT

Savolni diqqat bilan o‘qing

Select two answers — aynan 2 ta javob belgilang.

Select the best answer — 1 ta eng to‘g‘ri javob.

Kod berilsa — uni qadam-baqadam o‘zingiz bajarib ko‘ring.